# Sophia Tsang

609-955-2453 | xt73@cornell.edu | sophtsang.github.io | github.com/sophtsang | linkedin.com/in/xueqing-tsang

## Education

**Cornell University**, College of Engineering **Expected May 2027**
*Bachelor of Science in Computer Science, Minor in Robotics* **GPA: 3.923**
***Relevant Courses:*** Frontiers of Computer Vision, Intro to ML, Foundations of Robotics, Robot Learning, Robot Perception, Intro to Game Architecture, Analysis of Algorithms, Formal Verification, Differential Equations, Discrete Mathematics

## Experience

**MIT Lincoln Laboratory** **Lexington, MA**
**Software Engineering and Robotics Intern** | Python, PyTorch, TensorFlow, Docker, ROS2, VIO **May 2026 – Aug 2026**
- Built an end-to-end 3D terrain reconstruction pipeline for UAV imagery, recovering 3D surface geometry from *limited-perspective aerial views in real time* and achieving **92.0% geometric accuracy** against LiDAR ground-truth terrain
- Adapted *EOGS satellite photogrammetry* approach to UAV imagery, combining RPC camera modeling with VIO-based sun estimation to incorporate *physically accurate shadow geometry* as an additional geometric cue for Gaussian splatting
- Optimized the real-time processing pipeline by leveraging multiprocessing to decouple ROS2 camera stream dataset updates from background depth estimation, enabling *non-blocking data ingestion and continuous, live 3D map generation*

**Cornell Electric Vehicles** **Ithaca, NY**
**Perception Lead, Autonomy Engineer** | Rust, Python, C++, PyTorch, ROS2, Docker, Cargo **Oct 2024 – Present**
- Engineered *C++/Rust LiDAR driver* with a calibrated sensor measurement model and IMU-based ego-motion compensation, decoding raw UDP packets and correcting range, angular, timing, and sensor-offset errors to produce *temporally consistent, world-aligned 3D point clouds*, **reducing static-obstacle drift** from **0.92m to 0.04m** for *downstream perception and planning*
- Optimized *two-layer-graph clustering* on **80k+ points** using azimuth-binning and range-adaptive thresholds, reducing scan latency to **20–30ms** and far-range over-merging from **99% to <10%** for real-time obstacle detection and mapping
- Developed *occlusion-tolerant multi-object tracking* and point-cloud-to-*occupancy-grid* mapping to maintain persistent obstacle IDs and map free space and obstacle regions across frames for longer-horizon motion planning

**Noah Snavely Research Group** **Roosevelt Island, NY**
**Research Intern** | PyTorch, COLMAP, CUDA, Gemini, GCP, Flask, Render, React, Three.js, WebGL **June 2025 – Jan 2026**
- Co-authored ECCV 2026 paper, "Honey, I Shrunk the Arc de Triomphe", on *metric-scale monocular geometry estimation*
- Developed the *geo-referencing pipeline* that scales SfM reconstructions to absolute metric scale by leveraging geo-tagged imagery, optimizing viewpoint selection to improve large-scale 3D scene reconstruction
- Expanded Doppelgangers++ into a vision-language network that performs *end-to-end 3D reconstruction*, designed to visually disambiguate between distinct, yet look-alike surfaces and serve as a *more robust alternative to classical SfM*

## Projects

**Kernel::BSOD: OS Simulator** | Rust, WebAssembly, JavaScript, React, Three.js, Cargo **Aug 2026 – Present**
- Built a Rust process-management subsystem compiled to WebAssembly, implementing process control block table, explicit process lifecycle state machine, and zombie-process reaping to manage application instances through OS-inspired process states
- Implemented a Hyprland-inspired binary space partitioning (BSP) window manager, integrating process lifecycle events with reducer-driven window state to coordinate process-to-window mappings, focus, and dynamic tiling

**Re:Color** | Java, LibGDX, Agile Project Management **Jan 2026 – May 2026**
- Designed a color-based gameplay system with 7 other developers, enabling players to recolor the world by painting and absorbing color from the 2D platformer environment, and shipped the game *live* at GDIAC Showcase to **400+ attendees**
- Developed core player systems by implementing a behavior tree for *player state management*, synchronizing animations and refining movement physics, and building a *custom paint splatter rendering engine*

## Skills

***Languages:*** Python, Rust, C/C++, C#, Java, Rocq, OCaml, JavaScript/TypeScript, SQL, HTML, CSS, Verilog
***Frameworks/Libraries:*** PyTorch, NumPy, React.js, Three.js, WebAssembly, Cargo, Flask, OpenCV, sqlite3, WebGL, Matplotlib
***Tools:*** Git, CI/CD, ROS2, Linux, Docker, VSCode, LibGDX, Unity, Render, GCP, Figma, Fusion 360, Slurm